



Central University of
Technology, Free State

FACULTY OF ENGINEERING, BUILT ENVIRONMENT
AND INFORMATION TECHNOLOGY



Comment Coding Standard

Revision 2a

Department of Information Technology

Compiled by:

**Mr. P. Kruger
2019**

Table of Contents

1. Word of welcome.....	3
2. Copyright ©.....	3
3. Program comments	3
3.1. Programmer and program comment header.....	3
3.2. Method (function) comment header	4
3.3. “End” comments.....	6
3.4. Property comments.....	7
3.5. Constructors In Classes.....	8
4. Conclusion.....	10

1. Word Of Welcome

This guide has been designed to introduce the coding standard relating to program comments. This standard reflects the minimum requirements, regardless of which programming language is used. In this guide, all examples are shown using Microsoft® Visual C#®.

2. Copyright ©

The copyright on this material is strictly reserved. Therefore the reproduction and photocopying of this guide or parts thereof without obtaining the necessary permission, is prohibited.

The distribution of the guide or parts thereof can lead to a claim for damages and is furthermore a criminal offense.

3. Program Comments

Program comments form part of the program code documentation. These comments must clearly explain what the different parts of the program do. Comments may also be included for a segment of code that does a more complicated operation that might require more explanation.

Comments form a critical part of program documentation, and helps shorten the time spent on debugging and maintaining programs.

3.1. Programmer And Program Comment Header

You must include programmer/program information in each and every source file (.cs file). Each source file must include the comment block at the beginning of the file, starting at line 1. The following is an **example** of how this should look in your programs:

```
// Programmer name : P Kruger
// Student nr      : 9411111
// Assignment nr    : Ex 5.5 a
// Purpose         : The purpose of this program is to print a table of
//                  the binary, octal, and hexadecimal equivalents of
//                  the decimal numbers in the range 1-256. No String
//                  formats are used to do the conversions.
```

This comment block indicates the programmer and program information.

If you work in a group:

- the student numbers of each member in the group must appear in each .cs file
- these student numbers must be separated by a semicolon(;) and may not contain any spaces
- eg. 9412212;9512344;20192983;209121212

The **Assignment nr** field must be taken from the **REQUIREMENTS/SPECIFICATIONS DOCUMENT**. The **Purpose** field must be kept as short as possible, but must still clearly explain the purpose of the program (**GET THE PURPOSE FROM THE REQUIREMENTS/SPECIFICATIONS DOCUMENT**). If the file contains a class or a code

module/library, then the **Purpose** field must reflect the purpose of the class or code module/library, and not the purpose of the program that will use the class or module.

Please note that this will be **different** for each assignment, but the **format will stay the same**. Programmer/program information is a vital part of program design and documentation.

3.2. Method (function) Comment Header

Each method (function) must include the following comment block immediately after the opening curly brace of the method (function) body:

```
//  
//Name           : string ToBinary(int intDecimal)  
//Purpose        : Converts the input parameter intDecimal to its  
//               equivalent binary number  
//Re-use         : None  
//Method Parameters : int intDecimal  
//               - decimal value that must be converted  
//Output Type    : string  
//               - the converted binary number as a string  
//
```

The **Name** field must be taken from the **REQUIREMENTS/SPECIFICATIONS DOCUMENT**.

Again, the **Purpose** field must be kept as short as possible, but must still clearly explain the purpose of the method (function) (**GET THE PURPOSE FROM THE REQUIREMENTS/SPECIFICATIONS DOCUMENT**).

The **Re-use** field indicates the **names** of all other methods (functions) that the current method (function) invokes (calls). These other methods (functions) do not include methods (functions) build into the programming language. In other words, if the current method (function) invokes (calls) other methods (functions) that are defined/coded as part of the current program, their names must be listed here.

For each method (function) parameter, the name, type and purpose must be specified under the **Method Parameters** field. Again, the purpose must be kept as short as possible, but must still clearly explain the purpose of the parameter (**GET ALL THIS INFORMATION FROM THE REQUIREMENTS/SPECIFICATIONS DOCUMENT**).

If the method (function) returns a value, the type and purpose of this return value must be specified under the **Output Type** field. Again, the purpose must be kept as short as possible, but must still clearly explain the purpose of the return value. If the return value, or possible range of values, indicates some sort of status, then the value or possible range of values must be given again stating the purpose of each one (**GET ALL THIS INFORMATION FROM THE REQUIREMENTS/SPECIFICATIONS DOCUMENT**). Output does **not** refer to the output generated by the method (function), such as screen output, and output to a file. If the method (function) returns no value, then **Output Type** is **None**.

As an example of the **Re-use** field, consider the following methods (functions) in a program:

```
public static int LinearSearch(int value)
{
    // Name           : int LinearSearch(int value)
    // Purpose         : Search for value in global itemArray
    // Reuse           : None
    // Input Parameters : int value
    //                  - the value to search for
    // Output Type      : int
    //                  -1: Failure, not found
    //                  >=0: Index position of value in global
    //                  itemArray
    int rc = -1;
    int lc = 0;
    bool found = false;

    while (lc < itemArray.Length && found == false)
    {
        if (itemArray[lc] == value)
        {
            rc = lc;
            found = true;
        } // end if
        lc++;
    } // end while

    return rc;
} // end method

public static int Delete(int param1)
{
    // Name           : int Delete(int param1)
    // Purpose         : Try to Delete param1 from global itemArray
    // Reuse           : LinearSearch()
    // Input Parameters : int param1
    //                  - the value to be deleted
    // Output Type      : int
    //                  0: Success, found and deleted
    //                  -3: Failure, not found
    //
    int rc = -3;
    int index = -1;

    index = LinearSearch(param1);
    if (index >= 0)
    {
        itemArray[index] = -1;
        rc = 0;
    } // end if

    return rc;
} // end method
```

3.3. “End” Comments

“End” comments are comments that are added to signal the end of a control structure (selection and loop), method (function), class, module, program, namespace, etc.

Consider the following code segment example:

```
// Programmer name : P Kruger
// Student nr      : 9411111
// Assignment nr    : Ex 5.5 a
// Purpose         : The purpose of this program is to print a table of
//                  the binary, octal, and hexadecimal equivalents of
//                  the decimal numbers in the range 1-256. No String
//                  formats are used to do the conversions.

namespace modNumbers
{
    class Program
    {
        static void Main(string[] args)
        {
            //
            //Name           : void Main(string[] args)
            //Purpose        : Main entry point
            //Re-use         : ToBinary()
            //Input Parameters : string[] args
            //                - command line arguments (not used)
            //Output Type     : None
            //

            string output = "Decimal\t" + "Binary\t\t" + "Octal\t" +
                           "Hexadecimal\n";

            bool next;

            int number, temporaryVariable;
            string binary, octal, hexadecimal;

            for (number = 1; number <= 256; ++number)
            {
                binary = "";
                octal = "";
                hexadecimal = "";

                binary = ToBinary(number);
                //.
                //.
                //.
                //.
            } // end for
            if (next == false)
            {
                //.
            } // end if
        }
    }
}
```

```

        {
            //
        } // end else
    } // end Main
} // end class Program
} // end modNumbers

```

Simply put, and using C#, every closing brace “}” must be followed by a comment line indicating what this closing brace indicates the end of.

3.4. Property Comments

To comment properties in C#, the same convention will be used as for methods. The **get** part of a property will be seen as similar to returning a value. The **set** part of a property will be seen as similar to receiving input via a method parameter.

Consider the following example:

Instance field:

```
private string firstName;
```

A property for this field with both a **get** and a **set**:

```

public string FirstName
{
    //
    //Name           : property string FirstName
    //Purpose         : Public property to give access to firstName
    //                field
    //Re-use          : none
    //Input Parameter : string value
    //                - new value for firstName field
    //Output Type     : string
    //                - value stored in the firstName field
    //
    get
    {
        return FirstName;
    } // end get
    set
    {
        firstName = value;
    } // end set
} // end property

```

However, if there is no instance field, but only an **automatic property**:

```

public string FirstName
{
    //
    //Name           : property string FirstName
    //Purpose        : Automatic public property to give access to
    //                corresponding compiler generated field
    //Re-use         : none
    //Input Parameter : string value
    //                - new value for corresponding compiler
    //                generated field
    //Output Type    : string
    //                - value stored in the corresponding compiler
    //                generated field
    //
    //
    get; set;
} // end property

```

CRITICALLY compare the differences between these two properties.

3.5. Constructors In Classes

Constructors must also be commented. The same convention will be used as for other methods.

Consider the following example:

```

public class Subject
{
    private string code;
    private int fee;

    public string Code
    {
        //
        //Name           : property string Code
        //Purpose        : Public property to give access to code
        //                instance field
        //Re-use         : none
        //Input Parameter : string value
        //                - new value for code field
        //Output Type    : string
        //                - value stored in the code instance
        //                field
        //
        get
        {
            return code;
        } // end get

        set
    }
}

```

```

{
    if (value.Length == 7)
    {
        code = value;
    } // end if
    else
    {
        code = "<NOTSET>";
    } // end else
} // end set
} // end property

public int Fee
{
    //
    //Name           : property int Fee
    //Purpose        : Public property to give access to fee
    //               instance field
    //Re-use         : none
    //Input Parameter : int value
    //               - new value for fee field
    //Output Type    : int
    //               - value stored in the fee instance
    //               field
    //
    get
    {
        return fee;
    } // end get

    set
    {
        if (value > 0)
        {
            fee = value;
        } // end if
        else
        {
            fee = -1;
        } // end else
    } // end set
} // end property

public Subject(string name, string code, int fee)
{
    //
    //Name           : Subject(string name, string code, int fee)
    //Purpose        : Overloaded constructor used to update
    //               instance fields via properties
    //Re-use         : None
    //Input Parameters : string name
    //               - new value for Name field
    //               string code

```

```
//          - new value for code field
//          int fee
//          - new value for fee field
//Output Type : none
//
Name = name;
Code = code;
Fee = fee;
} // end method
} // end class
```

4. Conclusion

To comment a program is of extreme importance for the documentation and self documentation process. Commenting using the above Comment Coding Standard is a simple iterative process of: **copy** → **paste** → **change**

ALL the information you will need to successfully comment your program will be supplied. It is then just a case of you taking the time to do the comments correctly as described in this document.

Please be reminded that it is your responsibility to ensure that your Visual Studio C# solution compiles/builds without any syntax errors **before** you submit. Submissions with syntax/compile/build errors will not be graded.

In closing:

**YOU HAVE NO EXCUSE NOT TO
GET FULL MARKS FOR YOUR
COMMENTS**